

1. Write a program to evaluate expressions using {{ and }}. Create a text box and change the background of text box when we give a color value.

Answer:

```
<!DOCTYPE html>
<html ng-app="">
<head>
  <title>AngularJS Expressions</title>
</head>

<body>

  <h2>AngularJS Expression Example</h2>

  Name:
  <input type="text" ng-model="name">

  <h3>Hello {{name}}</h3>

  <br><br>

  Enter Background Color:
  <input type="text" ng-model="bgColor" placeholder="red, yellow,
lightblue">

  <br><br>

  <input type="text" ng-style="{ 'background-color': bgColor }"
placeholder="Background changes">

  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>

</body>
</html>
```

2. Understand a module and program to include a directive and a controller inside a module from external js file.

Answer:

index.html:

```
<!DOCTYPE html>
<html ng-app="myApp">

<head>
  <title>AngularJS Module</title>
</head>

<body ng-controller="myController">

  <h2>{{message}}</h2>

  <my-directive></my-directive>
```

```

    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
    <script src="app.js"></script>

</body>
</html>

```

app.js:

```

var app = angular.module("myApp", []);

app.controller("myController", function($scope){
    $scope.message = "Welcome to AngularJS";
});

app.directive("myDirective", function(){
    return {
        template: "<h3>This is a Custom Directive.</h3>"
    };
});

```

3. Write a program to bind the data using ng-model directive. Use ng-init, ng-repeat and ng-include directives inside your program.

Answer:

index.html:

```

<!DOCTYPE html>
<html ng-app="myApp">

<head>
    <title>AngularJS Directives</title>
</head>

<body ng-controller="myController" ng-init="college='ABC College'">

    <h2>{{college}}</h2>

    Name:
    <input type="text" ng-model="student">

    <h3>Student Name: {{student}}</h3>

    <h3>Subjects</h3>

    <ul>
        <li ng-repeat="subject in subjects">
            {{subject}}
        </li>
    </ul>

    <div ng-include="'header.html'"></div>

```

```
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>
```

```
<script>
    var app = angular.module("myApp", []);

    app.controller("myController", function($scope) {
        $scope.subjects = [
            "Java",
            "Python",
            "AngularJS",
            "HTML"
        ];
    });
</script>
```

```
</body>
```

```
</html>
```

header.html:

```
<h2>This is included using ng-include.</h2>
```

- Directives used:
 - ng-model -> Two-way data binding.
- ng-init -> Initializes variables.
- ng-repeat -> Repeats HTML elements.
- ng-include -> Includes an external HTML file.

4. What is a scope and program with \$scope and \$rootScope.

Answer:

Scope is an object that connects the controller with the HTML (view). It stores data and functions used in the view.

- **\$scope** - Available only within a particular controller.
- **\$rootScope** - Available throughout the entire AngularJS application.

Program:

```
<!DOCTYPE html>
```

```
<html ng-app="myApp">
```

```
<head>
```

```
    <title>Scope Example</title>
```

```
</head>
```

```
<body>
```

```
<div ng-controller="firstCtrl">
```

```
    <h2>{{ $root.message }}</h2>
```

```
    <h3>{{ name }}</h3>
```

```
</div>
```

```
<div ng-controller="secondCtrl">
```

```
    <h2>{{ $root.message }}</h2>
```

```
    <h3>{{ city }}</h3>
```

```
</div>
```

```

<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></
script>

<script>

var app = angular.module("myApp", []);

app.run(function($rootScope) {
    $rootScope.message = "Welcome to AngularJS";
});

app.controller("firstCtrl", function($scope) {
    $scope.name = "Suman";
});

app.controller("secondCtrl", function($scope) {
    $scope.city = "Bangalore";
});

</script>

</body>
</html>

```

Things to know:

1. What are the benefits of AngularJS?

Answer:

1. Two-Way Data Binding

- Automatically synchronizes data between the model and the view.

2. MVC Architecture

- Separates application into Model, View, and Controller, making code easier to maintain.

3. Reusable Components

- Directives allow creating reusable HTML components.

3. Dependency Injection

- Makes testing and code management easier by injecting required services.

4. Single Page Application (SPA) Support

- Loads content dynamically without refreshing the entire page.

5. Powerful Directives

- Built-in directives like ng-model, ng-repeat, ng-if, and ng-include simplify development.

6. Form Validation

- Provides built-in validation for forms with minimal code.

7. Less Code

- Reduces boilerplate code compared to traditional JavaScript.

8. Easy Testing

- Supports unit testing and end-to-end testing.

9. Open Source

- Free to use and supported by a large developer community.